ML Pipelines - automation and CI/CD

Maciej Kępa

Roadmap

Data Community

01

AI/ML architecture - how it all fits together

Understand the full ML stack from data sources to production, including roles, cloud reference architectures, layers, tools, and a simple ETL + model demo.

02

Data Preparation - practical foundations

Learn practical techniques for building ML data pipelines, cleaning data, validating datasets, and preparing a dataset in PySpark

03

Feature Engineering – the art of extracting value from data

Discover how to create effective features, handle different types of data, apply encoding and normalization, monitor feature drift, and track features with Spark + MLflow.

04

ML Algorithms - the classical approach

Get familiar with core algorithms like regression, trees, and gradient boosting, understand classification, regression, clustering, their pros and cons, and try MLlib demos in Databricks.

05

Model Training in Practice

Learn how to split data, perform hyperparameter tuning, scale training, and train models with code and metrics using MLflow.

06

Deep Learning - leveling up

Understand neural network basics, choose between TensorFlow and PyTorch, leverage GPU/TPU scaling, and build a simple CNN with visualization.

07

ML Pipelines - automation and CI/CD

Build repeatable ML workflows with orchestration tools, CI/CD pipelines, model registry/versioning, and a Databricks pipeline demo.

08

MLOps - manage your ML solution

Explore ML lifecycle management with tracking, model monitoring, data drift handling, canary deployments, A/B testing, and monitoring demos.

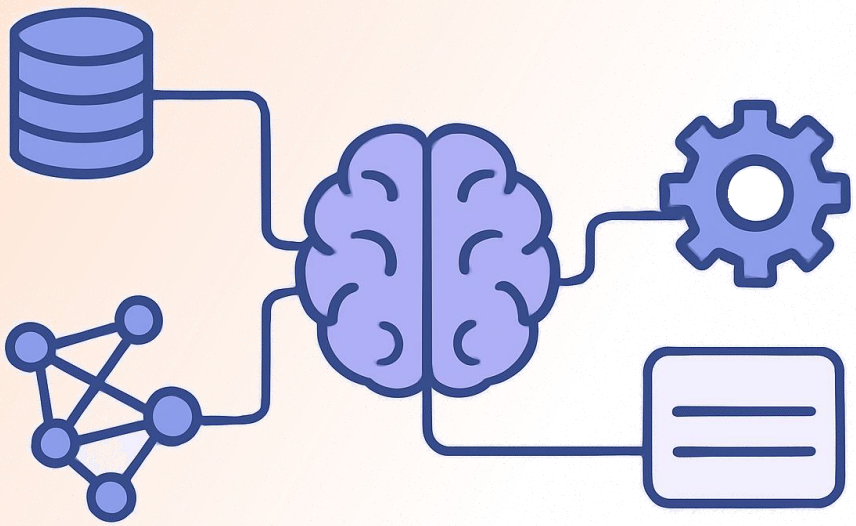
09

Generative AI and LLMs - the new wave of technology

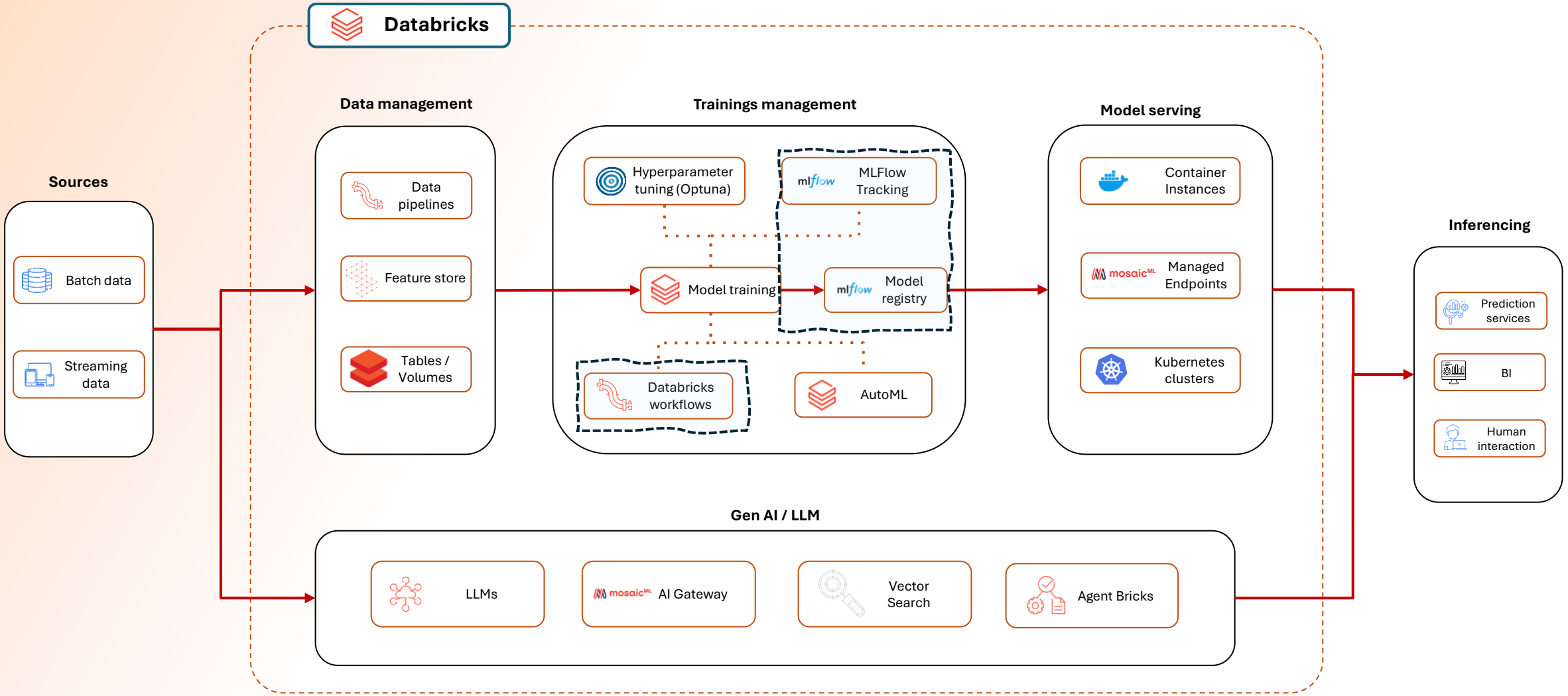
Learn how LLMs differ from classical ML, understand transformers, RAG, AI agent orchestration, and try a simple RAG demo in Azure OpenAI.

Agenda

1. Introduction
2. Orchestration methods
3. CI/CD for Machine Learning
4. Model registry & versioning
5. Workshop: Lakeflow Jobs orchestration
6. Workshop: CI/CD with Declarative Automation Bundles
7. Best practices



Introduction


Databricks
Sources
Batch data
Streaming data
Data management
Data pipelines
Feature store
Tables / Volumes
Trainings management
Hyperparameter tuning (Optuna)
MLFlow Tracking
Model training
Model registry
Databricks workflows
AutoML
Model serving
Container Instances
Managed Endpoints
Kubernetes clusters
Inferencing
Prediction services
BI
Human interaction
Gen AI / LLM
LLMs
AI Gateway
Vector Search
Agent Bricks

Reproducibility as a foundation

Ensuring reproducibility in Machine Learning projects requires implementing versioning mechanisms across multiple layers. Each layer must be controlled and auditable to recreate a model regardless of when or where it is executed.

Code

Source code, notebooks, and scripts versioned with Git. Includes model architecture, training logic, and inference code.

- Git commits and branches
- Notebook versioning
- Algorithm implementations

Data

Training datasets, feature stores, and validation sets with immutable snapshots. Critical for model lineage tracking.

- Dataset versioning
- Feature store snapshots
- Label provenance

Environment

Compute infrastructure, libraries, and dependencies. Ensures consistent execution across development and production.

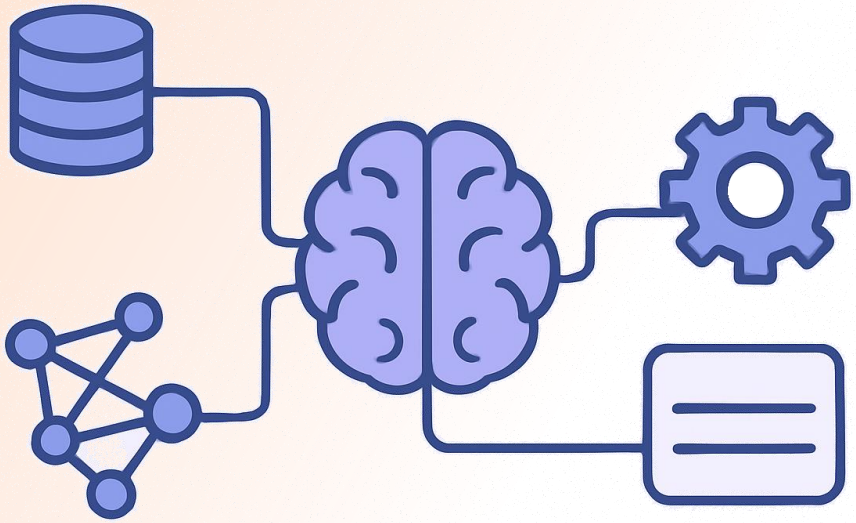
- Container images
- Library versions
- Hardware specifications

Configuration

Hyperparameters, pipeline settings, and deployment configurations. Separates code from runtime behavior.

- Hyperparameter tuning
- Pipeline parameters
- Feature engineering settings

This approach eliminates the 'works on my machine' problem and ensures that every team member can reproduce the exact same results.



Orchestration methods

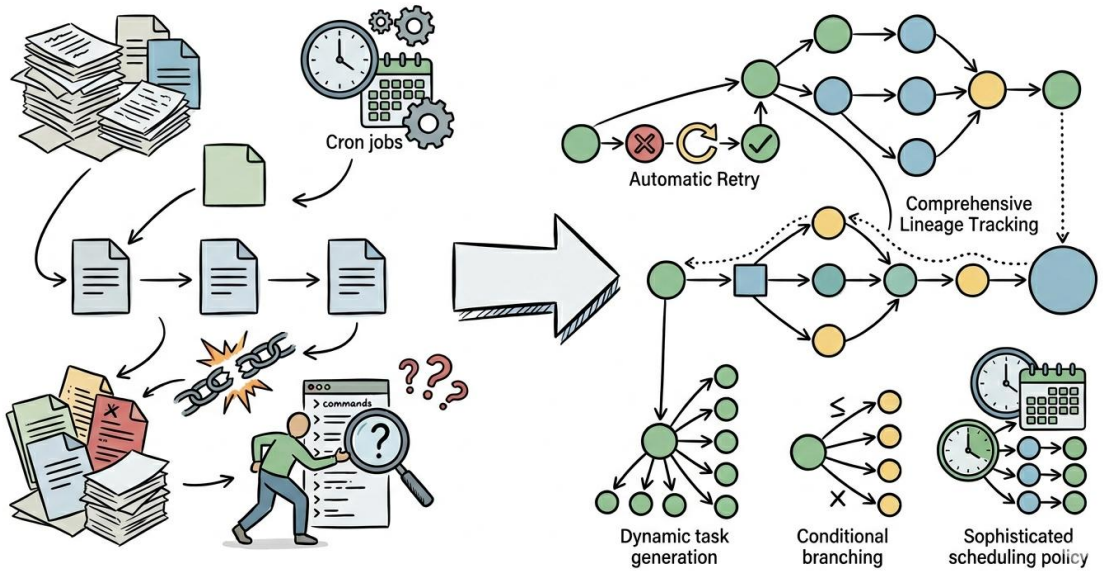
Evolution of ML orchestration

From Scripts to DAGs

Early ML workflows relied on cron jobs and shell scripts, creating brittle systems difficult to monitor and debug. The introduction of Directed Acyclic Graphs (DAGs) represented a paradigm shift, treating workflows as explicitly defined dependency graphs rather than sequential scripts.

DAG-based orchestration provides visual representations of pipeline dependencies, automatic retry logic for failed tasks, and comprehensive lineage tracking. Modern platforms extend this concept with dynamic task generation, conditional branching, and sophisticated scheduling policies.

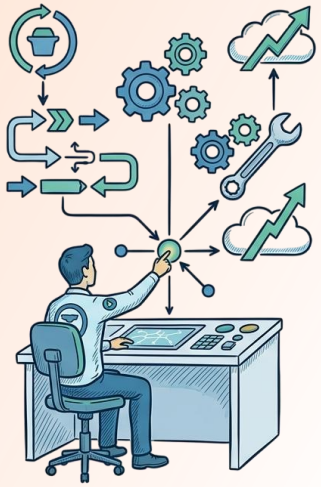
Today's orchestration landscape offers multiple solutions, from self-managed platforms requiring infrastructure expertise to fully managed, serverless options. The choice depends on organizational requirements, cloud strategy, and operational capabilities.

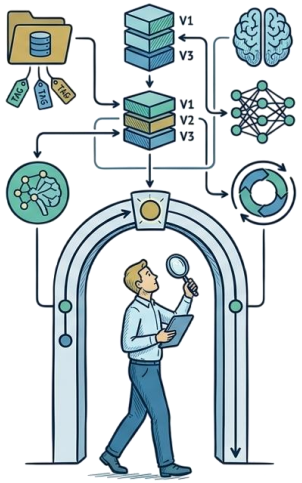


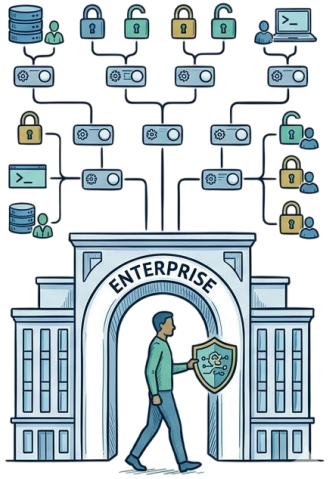
Vendor-specific orchestration

Data Community

Databricks Lakeflow Jobs represents Databricks native, orchestration solution designed specifically for the Lakehouse architecture. Unlike generic workflow engines, Lakeflow integrates directly with Unity Catalog and Delta Lake, providing seamless versioning and lineage tracking across the entire ML pipeline.







Fully Managed

Simple infrastructure provisioning and maintenance. Scales automatically based on pipeline complexity and resource requirements.

Ecosystem Integration

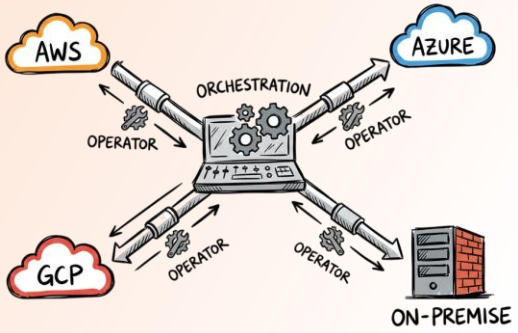
Native access to Unity Catalog for metadata, Delta Lake for data versioning, and MLflow for model tracking.

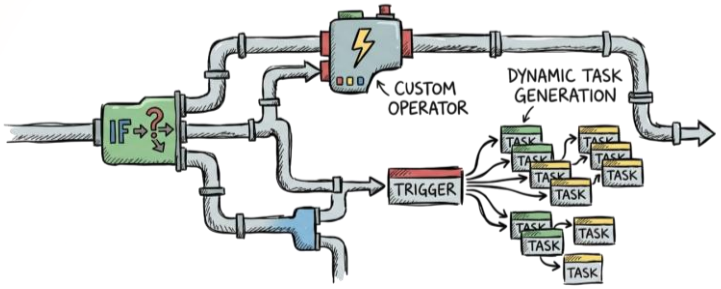
Enterprise Security

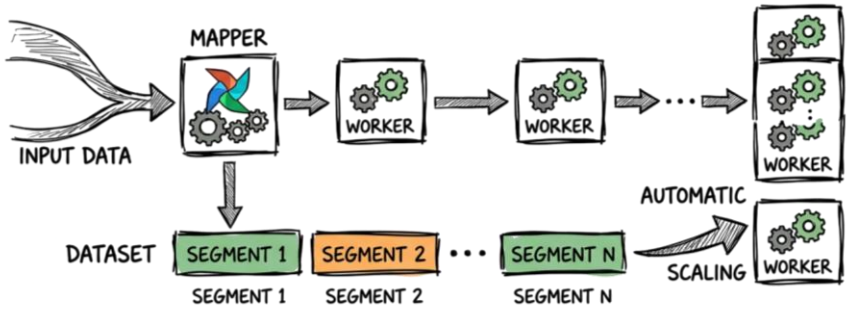
Inherits Databricks workspace permissions and supports fine-grained access control for each component.

Hybrid orchestration

Apache Airflow remains the industry standard for workflow orchestration, particularly in multi-cloud and hybrid environments. While Lakeflow excels within the closed Databricks ecosystem, Airflow provides unmatched flexibility for orchestrating workflows across disparate systems, cloud providers, and on-premise infrastructure.







Multi-Cloud Deployments

Coordinate ML pipelines that span AWS, Azure, GCP, and on-premise systems. Airflow operators exist for virtually every cloud service.

Custom Logic Requirements

Implement complex branching logic, dynamic task generation, and custom operators not available in managed solutions.

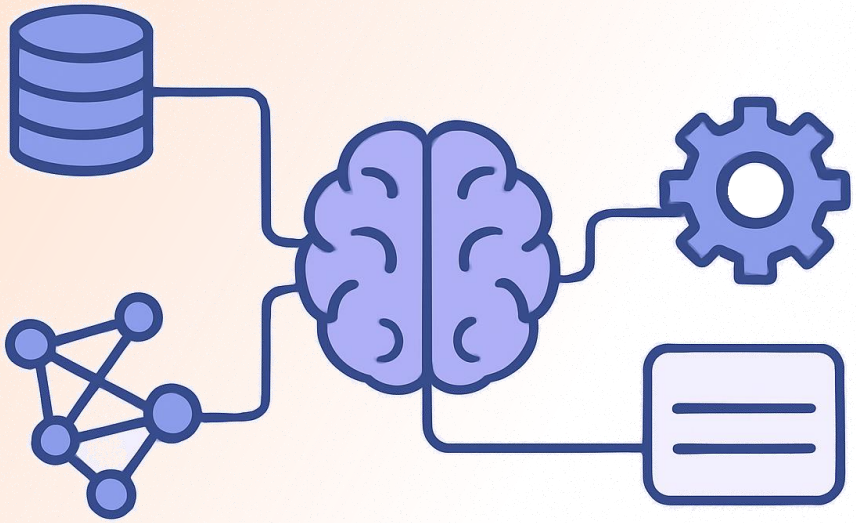
Dynamic Task Mapping

Automatically scales the number of parallel tasks at runtime based on data (e.g., training per-segment models).

How to choose?

The choice between the tools should be guided by a 'pragmatism-first' approach. If your processes occur primarily within the Vendor ecosystem, tool like Lakeflow Jobs offers lower operational overhead. However, if the pipeline must coordinate actions across multiple, unrelated platforms, Airflow remains the more mature solution.

Trait	Databricks Lakeflow Jobs	Apache Airflow
Operating model	SaaS / Serverless	Self-managed / Managed Service
Configuration	GUI + JSON/YAML (DABs)	Python (Code-first)
Dependency management	Build-in, visual DAGs	Programatically defined flows
Scaling	Automatic within the cloud	Horizontal scaling through workers
Costs	No extra costs (you pay only for compute)	Orchestrator instance costs + compute



CI/CD for Machine Learning

Infrastructure as Code for ML

What It Means

Treating infrastructure - servers, networks, and ML environments - as version-controlled software code, enabling consistent and reproducible deployments.

Why ML Needs It

ML systems depend on consistent environments across development, testing, and production. Infrastructure as code prevents environment drift, so models trained in dev behave the same way when deployed. It also makes it easier to manage changes safely, reproduce results, and keep deployments aligned across teams and stages.

Reproducibility

Same infrastructure, every time. Prevents environment drift so the same data, dependencies, and runtime behave identically from development to production.

Version Control

Track changes like code. Provides rollback paths, audit trails, and easier collaboration by showing exactly what changed, when, and by whom.

Automation

Eliminate manual provisioning errors and slow handoffs by replacing repetitive setup steps with repeatable code-driven deployment workflows.

Scalability

Grow infrastructure on demand. Code-driven provisioning lets teams add resources quickly and consistently without manual intervention as usage increases.

CI / CD / CT

Continuous Integration (CI)

Automating the merging of code changes into a shared repository, followed by automated builds and tests.

Why it matters: Catching bugs early saves time, reduces rework, and prevents broken code from reaching later stages of the pipeline.

Continuous Delivery (CD)

Automating the delivery of code changes to a staging or production environment after CI passes.

Why it matters: Automating deployment reduces human error, creates a more reliable release process, and speeds time-to-value for new features and fixes.

Continuous Training (CT)

Automating the retraining of ML models when new data becomes available or performance degrades.

Why it matters: Models must adapt as data changes to maintain accuracy, stay aligned with real-world conditions, and preserve business value over time.

⚠️ Although continuous training is frequently featured in modern literature, it remains uncommon in production environments.

Automating the ML Lifecycle

01

Git Push Trigger

Code changes in Git initiate the entire pipeline

02

Automated Unit Tests

Verify individual code components and model layers

03

Data Validation

Ensure data integrity, schema adherence, and statistical properties

04

Model Training

Automated training with performance metrics captured

05

Model Registry

Store and track trained models with versioning and metadata

06

Production Deployment

Automated deployment to validated environments

CI/CD Security

01

OIDC Federation

Use short-lived tokens instead of stored credentials. This reduces secret leakage risk and removes persistent passwords from pipelines.

02

RBAC Controls

Grant only the permissions a pipeline needs. Least privilege limits blast radius if access is misused.

03

Model Signatures

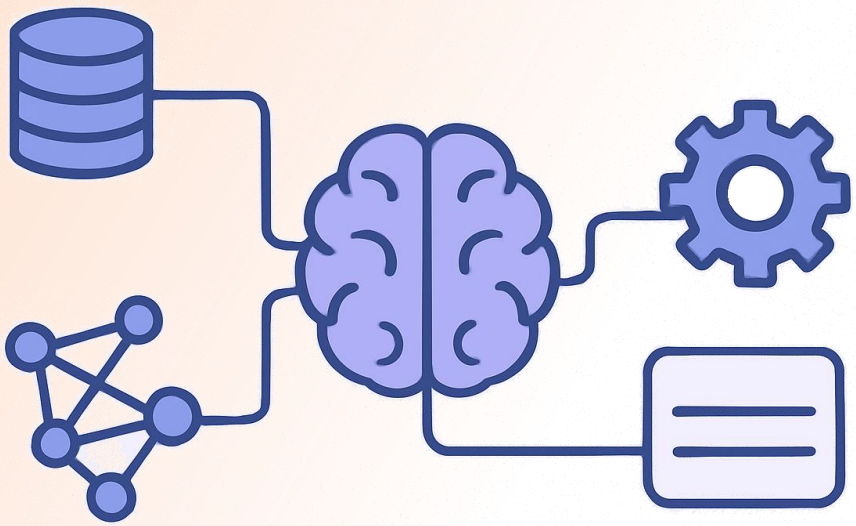
Validate input and output schemas before deployment. This catches mismatches early and prevents bad handoffs.

04

Secret Management

Store credentials centrally with rotation. This reduces sprawl and keeps secrets up to date.

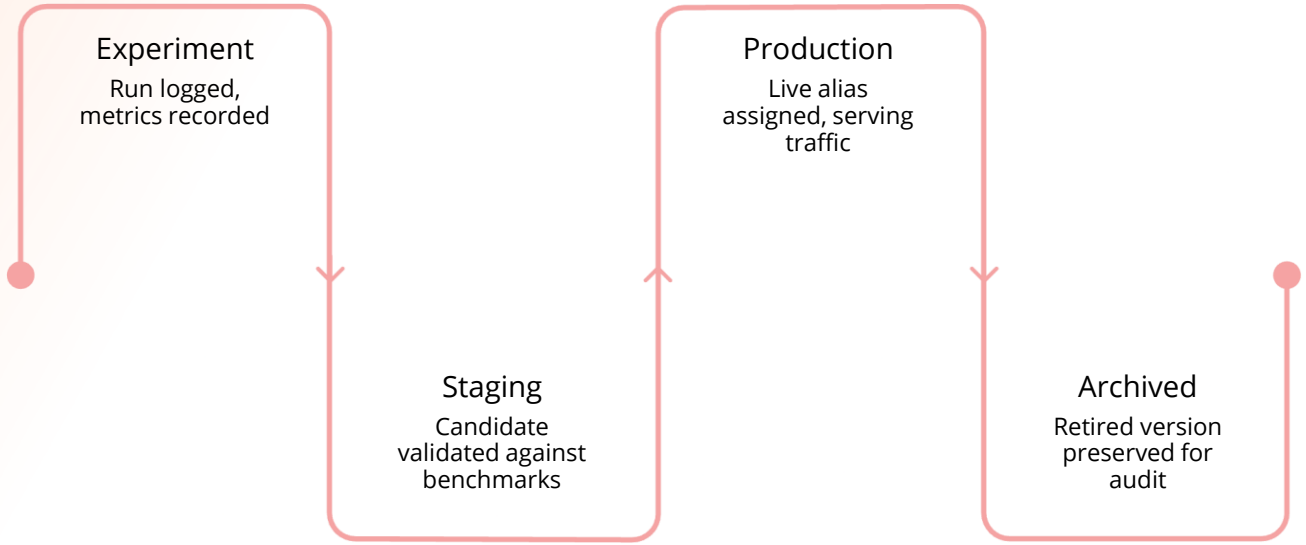
Application: Apply least privilege and secure access patterns so pipelines can provision infrastructure, deploy models, and access data stores without persistent credentials.



Model registry and versioning

Model registry

A model registry structures the model lifecycle through clearly defined stages. Rather than promoting artifacts informally, teams use explicit stage transitions - moving models from experimentation through validation and into production - with metadata and approval gates at each step.



Stage transitions integrate directly with CI/CD pipelines. Automated tests can trigger a promotion to staging; a human approval gate can gate the final push to production. Rollback is equally straightforward - reassign the production alias to any previous version and the serving infrastructure updates without redeployment.

Key capabilities of model registry

What the Registry Stores

- **Versioned artifacts** - serialized model files, preprocessing pipelines, and associated schemas, each bound to an immutable version number.
- **Tags & descriptions** - human-readable metadata describing the model's purpose, training dataset, and known limitations.
- **Metrics snapshots** - evaluation results captured at registration time, enabling direct cross-version comparison without re-running experiments.
- **Aliases** - mutable pointers like *@production* or *@champion* that decouple serving infrastructure from hard-coded version numbers.

Reference & Integration Patterns

Models can be referenced by name and version (e.g. `models:/MyModel/3`) or by alias (e.g. `models:/MyModel@production`). This makes downstream consumers - serving endpoints, batch jobs, evaluation scripts - resilient to version churn.

Registry operations should be embedded in CI/CD: registering on merge to main, promoting to staging on green test suite, triggering a review gate before production promotion.

> Aliases are the recommended pattern for production references. They let you roll back by updating a single pointer rather than redeploying consumers.

Governance through registry

The ultimate promise of a model registry is this: for any model version in production - past or present - you can answer exactly how it was built.

Code Provenance

Link each model version to the exact commit hash and training script revision. No ambiguity about which codebase produced a given artifact.

Data Lineage

Record the dataset version, split identifiers, and feature engineering logic alongside the model. Reproduce training from scratch at any point in history.

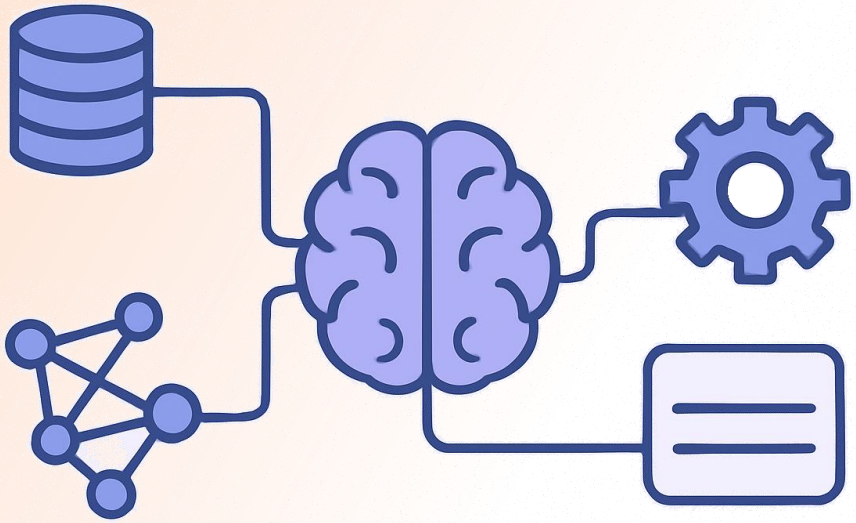
Audit Trail

Every stage transition, approval, and alias reassignment is logged with a timestamp and actor identity - satisfying compliance and incident investigation requirements.

Safe Rollback

When a production model degrades, reassign the alias to the last known-good version. The registry preserves all prior artifacts, so rollback is always available.

DEMO



Summary

Takeaways

Reproducibility is a „Layered" Requirement: In machine lerarning a result is only reproducible if you control all the distinct planes. If any layer is missing, you encounter the "works on my machine" problem.

Orchestration is the "Conductor" of the system: orchestration is about defining the relationship between tasks. It ensures that Task B only starts if Task A succeeds, handles automatic retries during errors, and schedules runs based on time or external events.

Treat Automation as a Quality Gate: CI/CD means that every change to the logic is automatically tested before it can touch the production environment. By using automated workflows to run unit tests and performance validations on every code update, you ensure that only "healthy" models are ever allowed to move forward.

Thank you!

Contact:

https://www.linkedin.com/in/maciej-kepa

https://github.com/maciejkepa/ai-ml-in-practice